

Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων

Τεκμηρίωση σχεδιασμού, υλοποίησης, εγκατάστασης και εκτέλεσης

Μάθημα	Τεχνολογίες Εφαρμογών Διαδικτύου
Τμήμα	Πληροφορικής & Τηλεπικοινωνιών
Εξάμηνο	ΣΤ' εξάμηνο σπουδών
Έτος	2026

Ομάδα εργασίας

Ονοματεπώνυμο	Αριθμός Μητρώου
[Συμπληρώστε ονοματεπώνυμο 1]	[AM 1]
[Συμπληρώστε ονοματεπώνυμο 2]	[AM 2]

Τίτλος εφαρμογής: EventHub

Τεχνολογική στοίβα: Angular 17, NestJS, TypeORM, MySQL/MariaDB, JWT, SSL/TLS

Πίνακας περιεχομένων

1. Εισαγωγή	3
2. Σύνοψη απαιτήσεων και λειτουργιών	3
3. Αρχιτεκτονική συστήματος	3
3.1 Δομή πηγαίου κώδικα	4
4. Σχεσιακό μοντέλο και δεδομένα	4
5. Ρόλοι, αυθεντικοποίηση και εξουσιοδότηση	4
6. Διαχείριση εκδηλώσεων	5
7. Αναζήτηση και σελιδοποίηση εκδηλώσεων	5
8. Μηχανισμός κρατήσεων	5
9. Messaging και ειδοποιήσεις	5
10. Εξαγωγή XML και JSON	6
11. Αλγόριθμος συστάσεων	6
11.1 Πηγές δεδομένων	6
11.2 Μοντέλο	6
12. Ενσωμάτωση dataset συστάσεων	6
13. Εγκατάσταση και εκτέλεση	6
13.1 Προαπαιτούμενα	6
13.2 Βάση δεδομένων	7
13.3 Backend	7
13.4 Frontend	7
13.5 Παραγωγική εκτέλεση	7
14. Παραδοχές και περιορισμοί	7
15. Έλεγχος λειτουργίας	7
16. Επίλογος	8

1. Εισαγωγή

Η παρούσα τεκμηρίωση συνοδεύει την εφαρμογή EventHub, η οποία αναπτύχθηκε για την υποχρεωτική εργασία του μαθήματος Τεχνολογίες Εφαρμογών Διαδικτύου. Στόχος της εργασίας είναι η υλοποίηση διαδικτυακής εφαρμογής για διαχείριση εκδηλώσεων, ηλεκτρονικές κρατήσεις εισιτηρίων/θέσεων, επικοινωνία χρηστών, εξαγωγή δεδομένων και προβολή εξατομικευμένων συστάσεων με Biased Matrix Factorization.

Στα επόμενα κεφάλαια παρουσιάζονται οι απαιτήσεις που καλύπτονται, η αρχιτεκτονική, οι βασικές σχεδιαστικές αποφάσεις, το σχεσιακό μοντέλο, οι ρόλοι χρηστών, ο μηχανισμός κρατήσεων, το messaging, ο αλγόριθμος συστάσεων, οι οδηγίες εγκατάστασης/εκτέλεσης και οι δυσκολίες που αντιμετωπίστηκαν.

2. Σύνοψη απαιτήσεων και λειτουργιών

Η εκφώνηση απαιτεί εφαρμογή web browser / web server με τέσσερις ρόλους: διαχειριστή, διοργανωτή, συμμετέχοντα και επισκέπτη. Η υλοποίηση καλύπτει εγγραφή/έγκριση χρηστών, σύνδεση με JWT, δημιουργία και διαχείριση εκδηλώσεων, αναζήτηση με φίλτρα, κρατήσεις με έλεγχο διαθεσιμότητας, messaging, εξαγωγή XML/JSON και συστάσεις εκδηλώσεων.

Απαίτηση	Υλοποίηση
SSL/TLS	Το backend εκκινεί με HTTPS όταν υπάρχουν πιστοποιητικά στο backend/ssl. Το Angular dev server επικοινωνεί μέσω proxy.
Ρόλοι	ADMIN, ORGANIZER, ATTENDEE και επισκέπτης χωρίς login για πλοήγηση/αναζήτηση.
Εκδηλώσεις	CRUD, DRAFT/PUBLISHED/CANCELLED, κατηγορίες, τύποι εισιτηρίων, φωτογραφίες και χάρτης OpenStreetMap.
Κρατήσεις	Έλεγχος χωρητικότητας και διαθέσιμων εισιτηρίων με συναλλαγή και pessimistic locking.
Messaging	Εισερχόμενα/απεσταλμένα, unread count, soft delete και ειδοποίηση σε ακύρωση εκδήλωσης.
Export	Εξαγωγή εκδηλώσεων σε XML συμβατό με τη δομή της εκφώνησης και αντίστοιχο JSON.
Recommendations	Biased Matrix Factorization από κρατήσεις, views και imported dataset interactions.

3. Αρχιτεκτονική συστήματος

Η εφαρμογή ακολουθεί client-server αρχιτεκτονική. Το frontend είναι Angular 17 standalone application με Angular Material, ενώ το backend είναι NestJS REST API με TypeORM. Η βάση δεδομένων είναι MySQL/MariaDB. Η επικοινωνία πραγματοποιείται μέσω HTTPS και τα προστατευμένα endpoints απαιτούν JWT.

Στρώμα	Τεχνολογίες / αρχεία
Frontend	Angular 17, Angular Material, Leaflet. Κύρια services: frontend/src/app/core/services.
Backend	NestJS modules: auth, users, events, bookings, messages, export, recommendations.
Persistence	TypeORM entities για users, events, ticket_types, bookings, messages, views, recommendation_interactions.

Στρώμα	Τεχνολογίες / αρχεία
Security	Passport JWT strategy, guards ρόλων, ValidationPipe με whitelist/transform.
Static assets	Φωτογραφίες στο backend/uploads και εξυπηρέτηση μέσω /uploads.

3.1 Δομή πηγαίου κώδικα

Το backend οργανώνεται ανά λειτουργική περιοχή. Τα entities βρίσκονται στο backend/src/entities, ενώ κάθε module έχει controller/service. Το frontend οργανώνεται σε core services/guards/interceptors και feature components για auth, home, events, bookings, messages και admin.

4. Σχεσιακό μοντέλο και δεδομένα

Το μοντέλο της βάσης ακολουθεί τις οντότητες της εκφώνησης και τις ανάγκες της εφαρμογής. Η εκδήλωση συσχετίζεται με διοργανωτή, κατηγορίες, τύπους εισιτηρίων, κρατήσεις, φωτογραφίες και προβολές χρηστών. Οι κρατήσεις συνδέουν συμμετέχοντα, εκδήλωση και τύπο εισιτηρίου.

Πίνακας / Entity	Ρόλος
users	Στοιχεία χρήστη, ρόλος, κατάσταση έγκρισης και optional external_dataset_id.
events	Βασικά στοιχεία εκδήλωσης, κατάσταση, χωρητικότητα, διοργανωτής και metadata dataset.
event_categories	Πολλαπλές κατηγορίες ανά εκδήλωση.
ticket_types	Τιμή, ποσότητα και διαθέσιμο υπόλοιπο ανά τύπο εισιτηρίου.
bookings	Κράτηση συμμετέχοντα, πλήθος εισιτηρίων, κόστος και status.
messages	Μηνύματα με sender/receiver, read_at και flags διαγραφής.
user_event_views	Ιστορικό προβολών για τον recommender.
recommendation_interactions	Imported ratings από dataset για τον BMF.
user_friends	Imported social links από dataset, διαθέσιμα για μελλοντική αξιοποίηση.

Παραδοχή: σε development χρησιμοποιείται TypeORM synchronize για ταχεία δημιουργία πινάκων. Σε παραγωγικό περιβάλλον θα έπρεπε να αντικατασταθεί από migrations.

5. Ρόλοι, αυθεντικοποίηση και εξουσιοδότηση

Η αυθεντικοποίηση γίνεται με username/password και JWT. Ο κωδικός αποθηκεύεται ως bcrypt hash. Ο αρχικός διαχειριστής δημιουργείται αυτόματα κατά την εκκίνηση αν δεν υπάρχει. Οι νέοι χρήστες εγγράφονται ως ATTENDEE με κατάσταση PENDING και απαιτείται έγκριση από ADMIN.

- JwtStrategy ελέγχει το token και ότι ο χρήστης είναι APPROVED.
- JwtAuthGuard προστατεύει endpoints που απαιτούν σύνδεση.
- RolesGuard περιορίζει λειτουργίες ανά ρόλο.
- OptionalJwtAuthGuard επιτρέπει public πρόσβαση σε event details, αλλά γράφει view όταν υπάρχει logged-in χρήστης.

6. Διαχείριση εκδηλώσεων

Οι διοργανωτές και οι διαχειριστές μπορούν να δημιουργούν εκδηλώσεις αρχικά ως DRAFT, να τις δημοσιεύουν, να τις τροποποιούν, να τις ακυρώνουν ή να τις διαγράφουν υπό προϋποθέσεις. Η εφαρμογή ελέγχει ότι το άθροισμα των ποσοτήτων εισιτηρίων δεν ξεπερνά τη συνολική χωρητικότητα.

- 1 Δημιουργία εκδήλωσης με τίτλο, κατηγορίες, τύπο, τόπο, ημερομηνίες, χωρητικότητα, περιγραφή και τύπους εισιτηρίων.
- 2 Δημοσίευση μόνο από κατάσταση DRAFT.
- 3 Επεξεργασία όσο η εκδήλωση δεν είναι CANCELLED.
- 4 Διαγραφή μόνο για DRAFT εκδηλώσεις χωρίς κρατήσεις.
- 5 Ακύρωση δημοσιευμένης εκδήλωσης χωρίς φυσική διαγραφή, ώστε να διατηρείται ιστορικότητα.

7. Αναζήτηση και σελιδοποίηση εκδηλώσεων

Η σελίδα αναζήτησης επιτρέπει φίλτρα τίτλου, κατηγορίας, πόλης, ημερομηνιών και τιμής. Τα query parameters περνούν από DTO με validation decorators, επειδή το global ValidationPipe χρησιμοποιεί whitelist. Χωρίς decorators τα φίλτρα απορρίπτονται, επομένως έγινε ρητή δήλωση των πεδίων στο EventFilterDto.

Η λίστα είναι σελιδοποιημένη τόσο στο frontend με MatPaginator όσο και στο backend με page/limit. Αντίστοιχη σελιδοποίηση υλοποιήθηκε για τις εκδηλώσεις διοργανωτή, τις κρατήσεις μιας εκδήλωσης και τις προτεινόμενες εκδηλώσεις.

8. Μηχανισμός κρατήσεων

Η κράτηση γίνεται μέσα σε database transaction. Χρησιμοποιείται pessimistic locking στην εκδήλωση και στον τύπο εισιτηρίου ώστε να αποφευχθούν race conditions όταν πολλοί χρήστες προσπαθούν να κλείσουν ταυτόχρονα θέσεις. Ελέγχεται τόσο η διαθεσιμότητα του συγκεκριμένου ticket type όσο και η συνολική χωρητικότητα.

- Δεν επιτρέπεται κράτηση σε μη PUBLISHED ή CANCELLED εκδήλωση.
- Δεν επιτρέπεται ο διοργανωτής να κάνει κράτηση στη δική του εκδήλωση.
- Μετά την επιτυχή κράτηση μειώνεται το available του ticket type.
- Το total_cost υπολογίζεται από τιμή εισιτηρίου επί πλήθος εισιτηρίων.
- Σε ακύρωση εκδήλωσης οι σχετικές ενεργές κρατήσεις γίνονται CANCELLED.

9. Messaging και ειδοποιήσεις

Το σύστημα μηνυμάτων υποστηρίζει αποστολή, εισερχόμενα, απεσταλμένα, unread count, mark-as-read και soft delete. Τα μηνύματα δεν διαγράφονται φυσικά από τη βάση όταν ο χρήστης τα αφαιρεί από τα εισερχόμενα ή τα απεσταλμένα, αλλά σημειώνονται με deleted_by_receiver ή deleted_by_sender.

Σε περίπτωση ακύρωσης εκδήλωσης δημιουργούνται αυτόματα μηνύματα προς όλους τους μοναδικούς χρήστες που είχαν ενεργή κράτηση. Το μήνυμα αποστέλλεται με sender τον διοργανωτή και περιλαμβάνει αναφορά ότι η κράτηση ενημερώθηκε ως CANCELLED.

10. Εξαγωγή XML και JSON

Ο διαχειριστής μπορεί να εξάγει τις εκδηλώσεις σε JSON και XML. Η XML έξοδος παράγεται με xml2js και ακολουθεί τη δομή της εκφώνησης: EventID, Title, Category, EventType, Venue, Address, GeoLocation, TicketTypes, Bookings, Organizer, Status, Description και Media.

Η εξαγωγή φορτώνει τις απαραίτητες σχέσεις: categories, ticket_types, bookings, attendee, organizer και photos. Έτσι το αρχείο εξόδου είναι αυτοτελές και μπορεί να χρησιμοποιηθεί ως στιγμιότυπο των δεδομένων.

11. Αλγόριθμος συστάσεων

Οι προτεινόμενες εκδηλώσεις εμφανίζονται στην αρχική σελίδα του logged-in χρήστη. Το backend endpoint GET /recommendations δέχεται page και limit και επιστρέφει σελιδοποιημένο αποτέλεσμα. Ο αλγόριθμος Biased Matrix Factorization υλοποιείται από το μηδέν στο recommendations.service.ts.

11.1 Πηγές δεδομένων

Πηγή	Rating / χρήση
Booking	Rating 5, επειδή η κράτηση δείχνει ισχυρό ενδιαφέρον.
UserEventView	Rating 2, επειδή η επίσκεψη σελίδας δείχνει ασθενέστερο ενδιαφέρον.
RecommendationInteraction	Ratings 1..5 από το imported dataset.

11.2 Μοντέλο

Το μοντέλο υπολογίζει πρόβλεψη με τον τύπο $\hat{r}(u,i) = \mu + b_u + b_i + p_u \cdot q_i$, όπου μ είναι ο μέσος όρος, b_u και b_i είναι biases χρήστη και εκδήλωσης, ενώ p_u και q_i είναι latent factor vectors. Οι παράμετροι είναι $K=10$, learning rate 0.005, regularization 0.02 και 30 epochs SGD.

Αν ο χρήστης δεν έχει ιστορικό, ή αν δεν υπάρχουν αρκετά δεδομένα για να παραχθεί σύσταση, ενεργοποιείται fallback σε δημοφιλείς δημοσιευμένες εκδηλώσεις, ταξινομημένες κυρίως με βάση τον αριθμό κρατήσεων.

12. Ενσωμάτωση dataset συστάσεων

Το dataset της εργασίας δεν αντιγράφεται στο repository, επειδή τα CSV είναι μεγάλα. Αντί αυτού δημιουργήθηκε streaming importer στο backend/scripts/import-rel-events-dataset.ts. Ο importer διαβάζει users.csv, events.csv, event_interest.csv, event_attendees.csv και user_friends.csv από DATASET_DIR και εισάγει ελεγχόμενο δείγμα στη βάση.

Παράδειγμα εκτέλεσης importer:

```
cd backend
npm run import:dataset
```

Οι imported χρήστες δημιουργούνται ως dataset_user_<external_id> με password από DATASET_PASSWORD. Τα imported events αποθηκεύουν external_dataset_id ώστε να αποφεύγονται διπλοεγγραφές σε επαναληπτικό import.

13. Εγκατάσταση και εκτέλεση

13.1 Προαπαιτούμενα

- Node.js 18 ή νεότερο.
- npm 9 ή νεότερο.
- MySQL ή MariaDB.
- Self-signed SSL certificates στο backend/ssl ή εναλλακτικά μεταβλητές SSL_KEY_PATH και SSL_CERT_PATH.

13.2 Βάση δεδομένων

```
CREATE DATABASE ted2026 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

13.3 Backend

Στο backend αντιγράφεται το .env.example σε .env και συμπληρώνονται DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET, ADMIN_USERNAME και ADMIN_PASSWORD. Έπειτα εκτελούνται:

```
cd backend
npm install
npm run start:dev
```

13.4 Frontend

Το frontend χρησιμοποιεί proxy /api προς το HTTPS backend. Για development εκτελούνται:

```
cd frontend
npm install
npm start
```

13.5 Παραγωγική εκτέλεση

Για παραγωγική εκτέλεση μπορούν να δημιουργηθούν builds με npm run build σε backend και frontend. Το nginx.conf του repository μπορεί να χρησιμοποιηθεί ως βάση reverse proxy με SSL/TLS.

14. Παραδοχές και περιορισμοί

- Δεν υλοποιείται πραγματική πληρωμή. Η κράτηση θεωρείται επιβεβαιωμένη μετά την οριστική υποβολή.
- Τα uploaded αρχεία φωτογραφιών αποθηκεύονται τοπικά στον φάκελο uploads.
- Το TypeORM synchronize χρησιμοποιείται για development και όχι ως βέλτιστη πρακτική παραγωγής.
- Ο recommender εκπαιδεύεται on demand στο request, επαρκές για την εργασία και μικρό/μεσαίο δείγμα, αλλά σε μεγάλη παραγωγική εγκατάσταση θα ήταν προτιμότερη περιοδική προεκπαίδευση/caching.
- Στο εξώφυλλο έχουν τοποθετηθεί placeholders για ονόματα ομάδας και αριθμούς μητρώου, επειδή δεν δόθηκαν στο workspace.
- Το dataset εισάγεται δειγματοληπτικά με μεταβλητές DATASET_MAX_* ώστε να αποφεύγεται υπερβολικός χρόνος import σε τοπικό περιβάλλον.

15. Έλεγχος λειτουργίας

Κατά την ανάπτυξη εκτελέστηκαν επαναλαμβανόμενα builds σε backend και frontend. Ελέγχθηκαν επίσης ειδικές ροές όπως σελιδοποίηση, αναζήτηση με query DTO, ακύρωση εκδήλωσης με ενημέρωση κρατήσεων/μηνυμάτων και fallback συστάσεων.

Έλεγχος	Ενδεικτική ενέργεια
Build backend	cd backend && npm run build
Build frontend	cd frontend && npm run build
Αναζήτηση	GET /events?title=Dataset%20Event%205&limit;=12
Συστάσεις	GET /recommendations?page=1&limit;=6 ως authenticated χρήστης
Ακύρωση	PATCH /events/:id/cancel και έλεγχος bookings/messages

16. Επίλογος

Η ανάπτυξη της εφαρμογής κάλυψε το πλήρες φάσμα μιας σύγχρονης web εφαρμογής: σχεδιασμό σχεσιακής βάσης, REST API, αυθεντικοποίηση και εξουσιοδότηση, responsive frontend, κρατήσεις με έλεγχο συνέπειας, messaging, εξαγωγή XML/JSON και αλγόριθμο συστάσεων. Ιδιαίτερη προσοχή δόθηκε στη διατήρηση ιστορικότητας των δεδομένων και στην αποφυγή ασυνεπειών στις κρατήσεις.

Οι κυριότερες δυσκολίες αφορούσαν την ενσωμάτωση του μεγάλου dataset, τη σωστή σελιδοποίηση σε πολλά σημεία, την ασφαλή ενημέρωση χωρητικότητας σε συνθήκες ταυτόχρονων κρατήσεων και την αλληλεπίδραση του ValidationPipe με τα query φίλτρα. Αντιμετωπίστηκαν με streaming import, backend pagination, συναλλαγές με pessimistic locking και ρητή δήλωση DTO decorators για τα query parameters.

Ως μελλοντικές επεκτάσεις προτείνονται migrations αντί για synchronize, caching ή offline training για τον recommender, καλύτερη διαχείριση media σε object storage, πλήρης audit logging και εμπλουτισμός των analytics των εκδηλώσεων.